SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	*Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.*		
Student Name:	S. Dharshini	**Reg. No.:**	192511260
Section / Batch:	2025 - CSE	**Date:**	26/08/2026

Code of Conduct

I S. Dharshini (192511260) *(Name / Reg No)* certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Dhar-.

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
	GRAND TOTAL:		**100 Marks**

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
   a) Clustering effect
   b) Memory usage
   c) Performance under high load factor
   d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
   a) Clustering issues
   b) Collision resolution efficiency
   c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
   a) Practical significance
   b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
   a) Space complexity (stack usage)
   b) Ease of implementation
   c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Name: S. Dharshini
Reg.No: 192511260
Course: Data Structures
Course code: - CSN0305

1. LINEAR PROBING Vs SEPERATE CHAINING.

Parameter	Linear Probing	Seperate chaining
Collision handling	Searches next available slot sequentially	Stores collided elements in a linked list / bucket.
Clustering	primary clustering occurs	No primary clustering.
Memory usage	Low, uses table slots only	Higher, requires extra memory for nodes / pointers.
Load factor	performance decreases significantly when load factor is high	Performs better at high load factors.
Search	Average O(1), worst O(n)	Average O(1), worst O(n)
Insertion	Average O(1), worst O(n)	Average O(1), worst O(n)
Deletion	More complicated because of probing	Easier.
Cache performance	Excellent due to contiguous memory	Usually poorer due to pointer traversal
Implementation	Simple	Relatively more complex

2. Quick Sort Vs Merge Sort for 10 Million Linked - List Nodes

Parameter	Quick Sort	Merge Sort
Collision method	Uses quadratic probe sequence	Uses second hash function
Formula	$(h(k) + i^2) \bmod m$	$(h1(k) + i \times h2(k)) \bmod$
primary clustering	Avoids	Avoids
secondary clustering	present	very low
collision resolution	Good	Better
Dense tables	performance decreases	Generally performs better
Complexity	Average O(1), worst O(n)	Average O(1), worst O(n)
Complexity of implementation	Simple	More complex
Number of Hash functions	one	two

3. Quadratic Probing Vs Double Hashing

Parameter	Quadratic Probing	Double Hashing
Collision method	Uses quadratic probe sequence	Uses second hash function
Formula	$(h(k) + i^2) \bmod m$	$(h1(k) + i \times h2(k)) \bmod m$

primary clustering	Present	Very low
Collision resolution	Good	Better
Dense tables	performance decreases	Generally performs better
Complexity	Average O(1), worst O(n)	Average O(1), worst O(n)
Complexity of implementation	Simple	More complex
Number of hash function	One	Two

4. BEST CASE VS WORST-CASE TIME COMPLEXITY OF SORTING ALGORITHMS

Algorithm	Best Case	Worst Case
Bubble sort	O(n)	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$
Insertion sort	O(n)	$O(n^2)$
Merge sort	O(n log n)	O(n log n)
Quick sort	O(n log n)	$O(n^2)$
Heap sort	O(n log n)	O(n log n)

5. RECURSIVE Vs ITERATIVE QUICK SORT

Parameter	Recursive Quick sort	Iterative Quick sort
Implementation	Simple & natural	More complex
Stack Usage	Function call stack	Explicit stack / data structure
Average space	O(log n)	O(log n) with controlled stack
Worst-case space	O(n)	Can be controlled
Stack overflow	Possible	Lower risk if explicitly managed
Readability	Easier	More complicated
Partitioning	Same	Same
Average time	O(n log n)	O(n log n)
Worst-case time	O(n²)	O(n²)